

Sắp Xếp & Thư Viện Chuẩn STL (Sorting & STL)

Chương 1 • Bài 8 • Khóa Học Lập Trình Thi Đấu Cơ Bản THPT

NỘI DUNG BÀI HỌC

- Phần 1: Giới thiệu hàm sắp xếp tối ưu `std::sort` trong thư viện `<algorithm>`.
- Phần 2: Sắp xếp tăng dần, giảm dần và viết hàm so sánh tùy biến (Custom Comparator).
- Phần 3: Cấu trúc `pair` và sắp xếp nhiều tiêu chí.
- Phần 4: Các hàm tìm kiếm nhị phân có sẵn: `lower_bound`, `upper_bound`, `binary_search`.
- Phần 5: Các tiện ích STL thường dùng: `reverse`, `unique`, `min_element`, `max_element`.

1. Hàm sắp xếp `std::sort`

Hàm `std::sort` trong C++ sử dụng thuật toán lai Introsort (kết hợp Quick Sort, Heap Sort và Insertion Sort) với độ phức tạp cam kết tối ưu $O(N \log N)$ cho mọi trường hợp dữ liệu, nhanh hơn hàng chục lần so với Bubble Sort hay Selection Sort.

[C++]

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int a[] = {5, 2, 9, 1, 5, 6};
    int n = 6;

    // Sắp xếp mảng thường tăng dần
    sort(a, a + n);

    // Sắp xếp vector giảm dần
    vector<int> v = {5, 2, 9, 1, 5, 6};
    sort(v.begin(), v.end(), greater<int>());

    return 0;
}
```

2. Hàm so sánh tùy biến (Custom Comparator)

Khi cần sắp xếp một danh sách theo tiêu chí đặc thù (ví dụ: số chẵn đứng trước số lẻ, điểm cao đứng trước nhưng nếu bằng điểm thì ưu tiên tên theo thứ tự từ điển), ta tự định nghĩa hàm so sánh bool `cmp(a, b)`.

[C++]

```

struct Student {
    string name;
    int score;
};

// Trả về true nếu a đứng trước b theo thứ tự mong muốn
bool cmp(const Student &a, const Student &b) {
    if (a.score != b.score) {
        return a.score > b.score; // Điểm cao hơn xếp trước
    }
    return a.name < b.name; // Cùng điểm: sắp theo tên từ A-Z
}

// Gọi: sort(students.begin(), students.end(), cmp);

```

3. Tìm kiếm nhị phân với STL

Điều kiện bắt buộc: Mảng hoặc vector phải được sắp xếp tăng dần trước khi gọi các hàm tìm kiếm nhị phân! Độ phức tạp chỉ $O(\log N)$.

Hàm	Cú pháp C++	Ý nghĩa / Giá trị trả về
<code>binary_search</code>	<code>binary_search(v.begin(), v.end(), x)</code>	Trả về true/false xem x có tồn tại hay không
<code>lower_bound</code>	<code>lower_bound(v.begin(), v.end(), x)</code>	Trả về con trỏ/iterator tới phần tử đầu tiên $\geq x$
<code>upper_bound</code>	<code>upper_bound(v.begin(), v.end(), x)</code>	Trả về con trỏ/iterator tới phần tử đầu tiên $> x$

[C++]

```

// Đếm số lần xuất hiện của x trong mảng đã sắp xếp:
int count_x = upper_bound(v.begin(), v.end(), x) - lower_bound(v.begin(), v.end(), x);

// Tìm vị trí chỉ số (0-based) của phần tử đầu tiên  $\geq x$ :
int idx = lower_bound(v.begin(), v.end(), x) - v.begin();

```

QUY TẮC SỐNG CÒN KHI DÙNG STL SORT:

- Quy tắc Bắt buộc của Hàm Comparator: Hàm so sánh `cmp(a, b)` phải tuân thủ nghiêm ngặt tính chất Thứ tự nghiêm ngặt (Strict Weak Ordering). Đặc biệt: nếu `a == b` thì `cmp(a, b)` **BẮT BUỘC** phải trả về `false`. Nếu viết `a >= b` hoặc `a <= b`, hàm `std::sort` có thể lặp vô hạn hoặc truy cập vùng nhớ cấm gây Runtime Error!
- Tìm kiếm nhị phân trên tập chưa sắp xếp: Luôn gọi `sort()` trước khi dùng `lower_bound`/`upper_bound`, nếu không kết quả trả về sẽ hoàn toàn sai lệch.